Gliwice, 9.06.2026

Project Report

Movie service

Author: Jakub Irla

Tutor: Damian Kucharski

1. Introduction

The Movie Service is a program that simulates a movie database platform like Filmweb or IMDb. It allows for the management and evaluation of a database of movies. The system distinguishes between two types of accounts: administrators and regular users. Each account type operates under specific permissions:

1. All users must authenticate via a login and password system to access the service.
2. All users can view movies list.
3. Administrators can manage the database by adding new movies or deleting existing ones.
4. Regular users can interact with the movies by rating them on a scale of 1 to 10 and adding reviews.

These interactions are managed dynamically during the program's loop. Initially, a default administrator account is provided. To ensure data persistence between sessions, the application stores all user accounts, movie details, and user-created data locally using text files.

2. External Specification

The program is used to be operated through the terminal. All user interactions are managed through simple text-based menus directly in the console.

Compiling and Running

The program is compiled from the command line using a standard C++ compiler with the following syntax:

```
> g++ main.cpp Admin.cpp RegularUser.cpp Movie.cpp Database.cpp
User.cpp Utils.cpp -o movie_service
```

The program is executed from the command line with the following syntax:

Linux:

```
>./movie_service
```

Windows:

```
>movie_service.exe
```

Usage

During the operation of the program, the user is presented with numeric menus. The initial state requires the user to log in or register.

```
=== Movie service ===
1. Login
2. Register
3. Exit
Choice:
```

Upon successful authentication, the program transitions to the respective menu based on the user's role. Administrators are presented with options to manage the movie database.

```
Admin Menu:
1. Add Movie
2. View Movies
3. Delete Movie
4. Logout
Choice:
```

Regular users are presented with options to view the catalogue, submit ratings, and manage reviews.

```
User Menu:
1. View Movies
2. Rate Movie (change / remove existing)
3. Add Review (change / remove existing)
4. Check Movie Reviews
5. Logout
Choice:
```

After choosing an option, by inputting the appropriate number, the program can ask for additional information, as in example below.

```
Choice: 2
Enter movie index to rate: 1
Enter rating (1-10, or 0 to remove): 10
```

The output of users' interactions can be observed by accessing the data with the program and is saved in three separate text files: *users.txt*, *movies.txt*, and *userdata.txt*. These files serve as the persistent storage mechanism for the program.

```
Choice: 2
1. The Empire Strikes Back by George Lucas (1982) Avg Rating: 4
2. Star Wars: Episode IV by George Lucas (1979) No ratings
```

```
The Empire Strikes Back|George Lucas|1982
Star Wars: Episode IV|George Lucas|1979
```

3. External Specification

The solution is implemented in the C++ programming language, utilizing object-oriented programming. The architecture is modular, divided into separate classes to handle distinct functionalities such as user interface, database management, etc.

Data structures

To implement the solution, the program utilizes vectors and maps.

- *std::vector<User*> users*: Stores pointers to base *User* objects, enabling switching between accounts, when using the program.
- *std::vector<Movie> movies*: Stores the current list of movies in memory.

Classes and functions

- *User*: This abstract class encapsulates the *username* and *password*. It defines virtual methods, whose implementations will differ in subclasses (e.g. *menu*), as well as common methods for all subclasses.
- *Admin* and *RegularUser*: These classes inherit from *User*. The *Admin* class implements the *menu* function to provide database management loops (*addMovie*, *deleteMovie*). The *RegularUser* class implements the *menu* function to handle catalog interactions (*rateMovie*, *addReview, viewMovieReviews*).
- *Movie*: This class represents a single entity within the movies list. It contains basic attributes and maintains lists of reviews and ratings for the movie. It includes functions such as *getAverageRating* to calculate scores dynamically based on the current data.
- *Database*: This class serves as the central data manager. Functions such as *loadMovies* and *saveMovies* handle the translation of the application's state to and from the text files.
- readIndex: utility function located in *Utils.cpp*, which ensures that terminal input is of an integer type. If a user provides invalid characters, it clears the input stream's errors and clears the buffer.

Stability

Significant attention was given to application stability. The implementation of input validation, safe file handling, checking parsing safety, and memory leak protection ensures the program will not crash due to improper save files and user input.

4. Testing

The program was tested manually by executing it from the terminal with various user inputs.

Test case 1: Valid Parameters (User Rating):

```
=== Movie service ===
1. Login
2. Register
3. Exit
Choice: 1
Username: jason
Password: jason

User Menu:
1. View Movies
2. Rate Movie (change / remove existing)
3. Add Review (change / remove existing)
4. Check Movie Reviews
5. Logout
Choice: 1
1. Star Wars: Episode IV by George Lucas (1979) Avg Rating: 4
2. The Empire Strikes Back by George Lucas (1982) Avg Rating: 6

User Menu:
1. View Movies
2. Rate Movie (change / remove existing)
3. Add Review (change / remove existing)
4. Check Movie Reviews
5. Logout
Choice: 2
Enter movie index to rate: 2
Enter rating (1-10, or 0 to remove): 10
Rating saved.

User Menu:
1. View Movies
2. Rate Movie (change / remove existing)
3. Add Review (change / remove existing)
4. Check Movie Reviews
5. Logout
Choice: 1
1. Star Wars: Episode IV by George Lucas (1979) Avg Rating: 4
2. The Empire Strikes Back by George Lucas (1982) Avg Rating: 8
```

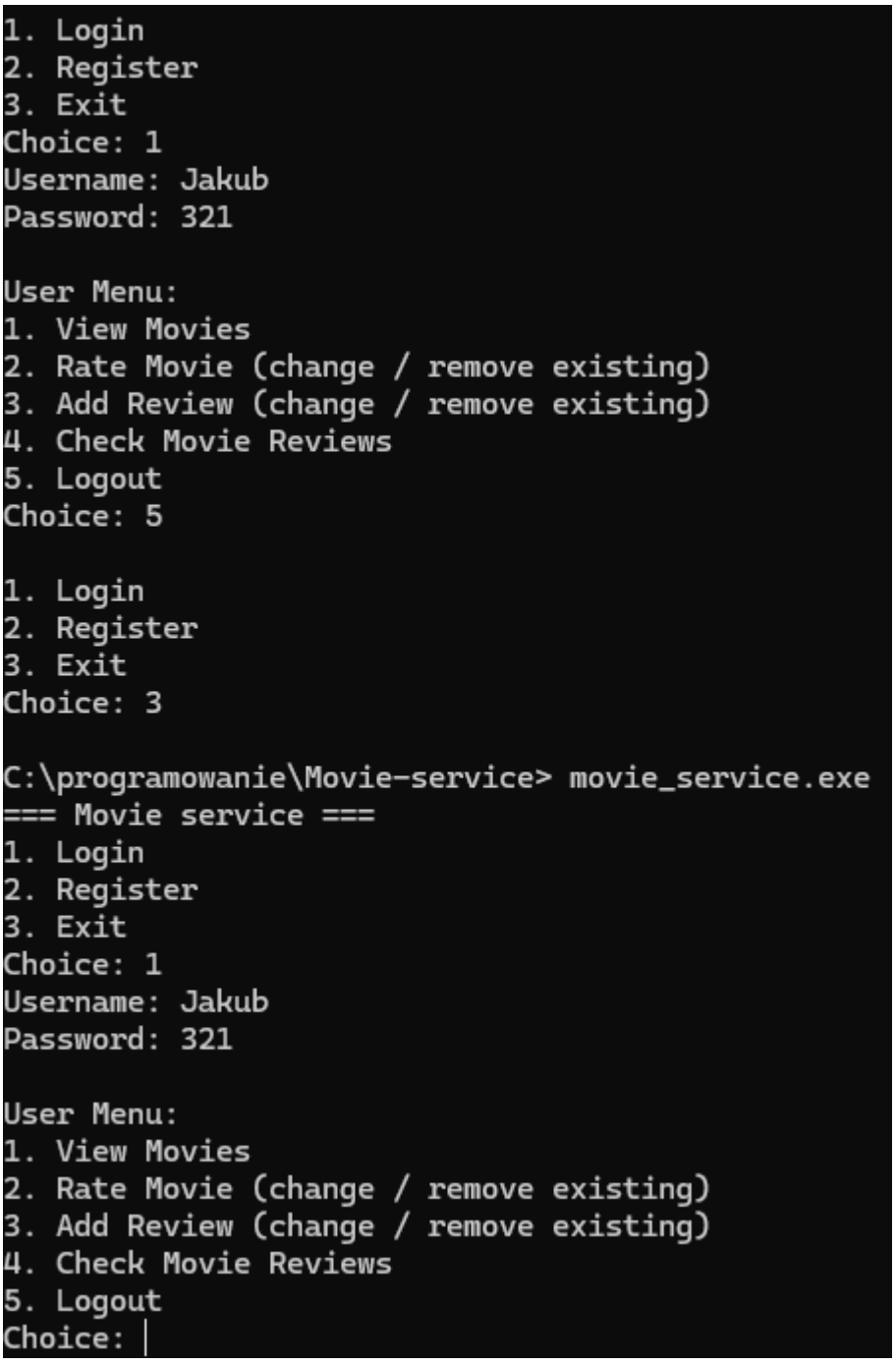
```
1|jason|10|
```

The program works correctly. The appropriate menu successfully loaded the *RegularUser* interface, and the rating was successfully recorded in memory and the *userdata.txt* file.

Test case 2: Registration and authentication mechanisms:

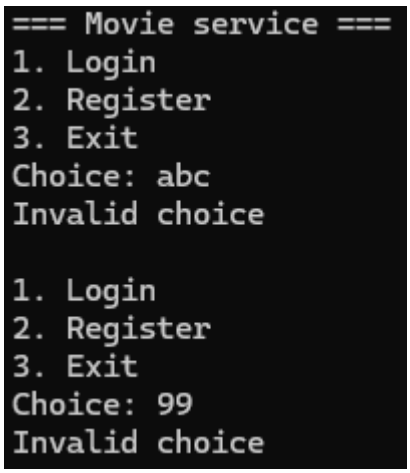
The registration and authentication mechanisms work correctly. The program successfully blocks unrecognized credentials, registers a new user, rejects incorrect passwords for existing accounts, and grants access to the user menu upon successful login.

Test case 3: Data persistence between sessions:



The program effectively persists data between sessions. After registering a user and exiting the application completely, restarting the program allowed the same user to log back in successfully, confirming that the *users.txt* file is correctly saved and loaded upon starting new session.

Test case 4: Invalid Parameters (Menu choice):



The program works correctly. The *readIndex* function intercepted the non-integer input, prevented an infinite loop error, and safely allowed the user to try again.

5. Conclusions

The implementation of the Movie Service successfully meets all specified requirements. The final project presents a console application that efficiently manages user authentication and movie data. By utilizing object-oriented programming, the system is well-structured and modular.

The separation of logic, file management, and user interface functions ensures the code can be easily modified. Furthermore, the correctness of the implementation and its stability against improper user input was consistently verified by the test cases.

For future work, several enhancements could be made to scale the project further. The current local text file database could be exchanged for an online database service to allow multiple users to interact with the application simultaneously across different devices. Additionally, the program could be improved by encrypting passwords to enhance security. Finally, adding a graphical user interface instead of a command-line interface would make interacting with the application much more pleasant for the user.